

W02 การพัฒนาเว็บแบบหน้าเดียวโดยใช้ React

Categories (หมวดหมู่)	Detailed Type (ประเภท)	Definition (คำอธิบาย)	Examples (ตัวอย่าง)
Monolithic Architecture	Monolithic	ระบบที่ทุกส่วนของแอปพลิเคชันเชื่อมโยงและจัดการในหน่วยเดียวทั้งหมด	WordPress, Shopify
Client-Server Architecture	Single Page Application (SPA)	แอปพลิเคชันที่โหลดหน้า HTML เพียงหน้าเดียว และอัปเดตเนื้อหาแบบไดนามิกโดยไม่ต้องโหลดหน้าใหม่	Gmail, Google Maps, Facebook
Client-Server Architecture	Multi-Page Application (MPA)	สถาปัตยกรรมแบบดั้งเดิมที่ทุกการกระทำต้องโหลดหน้าใหม่จากเซิร์ฟเวอร์	Wikipedia, เว็บไซต์อีคอมเมิร์ซ
Client-Server Architecture	Server-Side Rendering (SSR)	เซิร์ฟเวอร์สร้าง HTML ของหน้าเว็บทั้งหมดและส่งไปยังไคลเอนต์เพื่อแสดงผล	Next.js, Ruby on Rails
Client-Server Architecture	Client-Side Rendering (CSR)	เบราว์เซอร์ดาวน์โหลด HTML, CSS, และ JavaScript ขั้นพื้นฐาน และแสดงผลหน้าเว็บทั้งหมดในฝั่งไคลเอนต์	React, Vue.js
Client-Server Architecture	Two-Tier Architecture	การเชื่อมต่อระหว่างไคลเอนต์และเซิร์ฟเวอร์โดยตรงสำหรับการแลกเปลี่ยนข้อมูล	แอปพลิเคชันเดสก์ท็อปที่เชื่อมต่อฐานข้อมูลโดยตรง
Client-Server Architecture	Three-Tier Architecture	สถาปัตยกรรมแบบเลเยอร์ที่แยกไคลเอนต์, ลอจิกแอปพลิเคชัน, และฐานข้อมูลออกจากกันเพื่อความสามารถในการขยายและจัดการได้	Web apps using Java EE, .NET
Microservices Architecture	Microservices	แอปพลิเคชันที่ประกอบด้วยบริการย่อยที่เป็นอิสระซึ่งสื่อสารกันผ่าน API	Netflix, Amazon
Microservices Architecture	Serverless	การจัดการ Backend บนคลาวด์โดยไม่ต้องดูแลเซิร์ฟเวอร์เอง	AWS Lambda, Firebase Functions
Service-Oriented Architecture (SOA)	Service-Oriented	การให้บริการส่วนต่าง ๆ ผ่านเครือข่าย เพื่อสนับสนุนการใช้งานซ้ำและการทำงานร่วมกัน	eBay, PayPal
Service-Oriented Architecture (SOA)	Headless	ส่วน Frontend แยกออกจาก Backend โดยมี API เป็นตัวจัดการการเชื่อมโยง	Contentful, Shopify Headless API
Event-Driven Architecture	Event-Driven	แอปพลิเคชันตอบสนองต่อเหตุการณ์หรือการเปลี่ยนแปลงสถานะเพื่อการประมวลผลแบบเรียลไทม์	Slack, Uber
Event-Driven Architecture	API-First	มุ่งเน้นการพัฒนา API เป็นอันดับแรก และแอปพลิเคชันจะใช้งาน API เหล่านั้น	Postman, Swagger-based applications

1 การจัดเตรียมสภาพแวดล้อม

นักศึกษาต้องเตรียมพร้อมให้เครื่อง และตนเองมีสิ่งต่อไปนี้

1. Node.js (เวอร์ชัน 14.18+ หรือสูงกว่า)
2. Git ติดตั้งในเครื่อง
3. บัญชี GitHub
4. VS Code

2 ขั้นตอนที่ 1: สร้างโปรเจกต์ Vite ใหม่

1. เปิด terminal และรันคำสั่ง

```
npm create vite@latest my-react-spa -- --template react
cd my-react-spa
```

2. ติดตั้ง dependencies:

```
npm install
```

3 ขั้นตอนที่ 2: จัดโครงสร้างโปรเจกต์

1. สร้างโฟลเดอร์และไฟล์ให้โครงสร้างโปรเจกต์มีลักษณะดังนี้

```
my-react-spa/
├── node_modules/
├── public/
├── src/
│   ├── assets/
│   ├── components/
│   ├── pages/
│   ├── App.jsx
│   └── main.jsx
├── index.html
├── package.json
└── vite.config.js
```

2. สร้างไดเรกทอรีที่จำเป็น:

```
mkdir src/components src/pages src/utls
```

4 ขั้นตอนที่ 3: ติดตั้ง Dependencies ที่จำเป็น

ติดตั้ง React Router และแพ็คเกจที่จำเป็นอื่นๆ:

```
npm install react-router-dom @vitejs/plugin-react
```

5 ขั้นตอนที่ 4: ตั้งค่า Vite

อัปเดตไฟล์ vite.config.js:

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

// https://vitejs.dev/config/
export default defineConfig({
  plugins: [react()],
  base: '/my-react-spa/', // แทนที่ด้วยชื่อ repository ของคุณ
})
```

6 ขั้นตอนที่ 5: ตั้งค่า React Router

1. สร้างหน้าพื้นฐานใน src/pages/:

```
src/pages/Home.jsx:
const Home = () => {
  return (
    <div>
      <h1>ยินดีต้อนรับสู่ SPA ของฉัน</h1>
      <p>นี่คือหน้าแรก</p>
    </div>
  )
}

export default Home
```